ROHM ML63Q2557（DT-EBML63Q2557）× Solist-AI™ 技術解説ドキュメント（全5章）

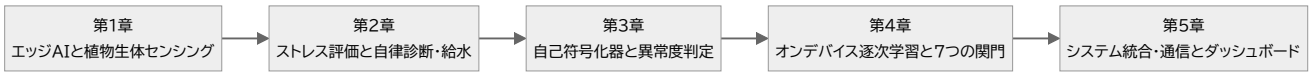
Plant Doctor 生体AI・オンデバイス学習 解説書

植物の「いつも」をその場で覚え、異常のときは覚えないエッジAI — センシング・ストレス評価・自己符号化器・7つの関門・システム統合

本書は、ROHM のマイコン **ML63Q2557**（DT-EBML63Q2557 ボード）とオンデバイス学習アクセラレータ **Solist-AI™** を使った植物見守りシステム「**Plant Doctor**」の技術解説書です。植物の「いつも」を鉢のそばで覚え、いつもと違う兆候を捉えて水やりと経過観察まで行う仕組みを、ファームウェアのソースコードと設定値、実機の記録にもとづいて詳しく解説します。

従来の自動水やり器は、あらかじめ決めた土壌水分のしきい値だけでポンプを動かしていました。しかし実際の栽培では、植物の種類や鉢の大きさ、土の保水性、窓際か照明下かによって「普段の乾き方」や「葉温の動き」が大きく異なります。Plant Doctor は、ルールによる安全管理（乾燥・熱・水やり失敗の検出と給水）と、Solist-AI による株ごとの「いつも」の学習を組み合わせることで、決定論的な安全性と個別の環境への適応を両立させています。

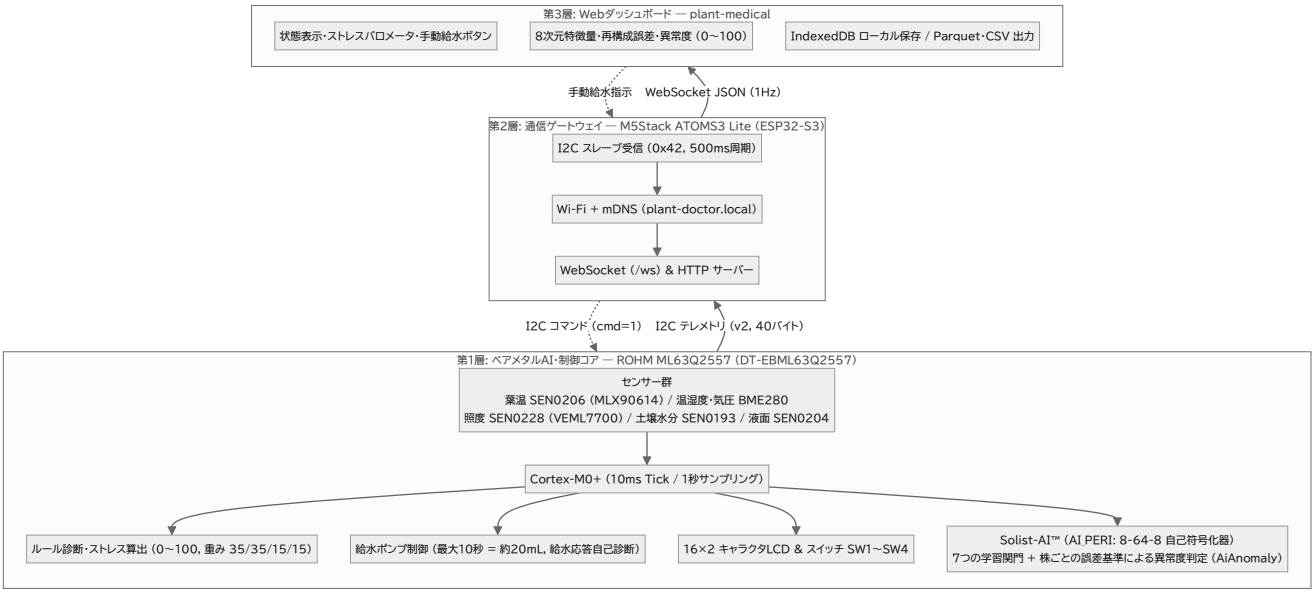
カリキュラム構成（全5章）



章	タイトル	主な内容
第1章	**エッジAIと植物生体センシング技術**	蒸散と気化熱による葉温・気温差（ΔT）の意味、5種類のセンサー（MLX90614・BME280・VEML7700・SEN0193・SEN0204）の取得処理と単位換算（照度の生値と 0.0576 lx 換算、土壌水分の2点校正）、ボード1枚で完結する3層構成
第2章	**多変量植物ストレス評価と自律診断アルゴリズム**	4要素（土壌35・熱35・日照15・湿度15）の重みと最大値支配（75%）によるストレスバロメータ（0〜100）、優先順位付きの原因候補推定、自律給水の安全条件と給水後（30秒・3分・5分）の自己診断、30分での給水失敗解除
第3章	**自己符号化器（Autoencoder）と多次元生体異常検知**	8次元特徴量の設計と `NormalizeToQ8` 正規化（変化速度の履歴15分待機と範囲設定）、8-64-8 自己符号化器の再構成誤差（ppm）、株ごとの平均・ばらつき学習による異常度（0〜100）算出、1回外れても1減らす加減算カウンタによる `WARNING` 判定
第4章	**オンデバイス逐次学習（ODL）の数学と7つの関門**	過去データを保存しない逐次最小二乗法（RLS）と忘却係数 $\lambda = 1.0$、学習フェーズ（Profiling / Stabilizing / Monitoring）、健康なモデルを汚さない7つの関門、給水後1時間半の休止、異常時の間引き（1分に1回）と3時間後の学び直し
第5章	**システム統合・通信テレメトリと生体コックピット**	I2C バイナリテレメトリ（v2、40バイト、生値照度、異常診断時の異常度85送信）、手動給水経路の統一（ `AppStateMachine_RequestManualWatering` ）、16×2 LCD 表示とデモモード（SW1長押し・ `M=` ）、ATOMS3 Lite と Web ダッシュボード `plant-medical`

システム全体像

Plant Doctor は、センサー取得から AI の学習・推論、ポンプ駆動までを単体で完結させる「マイコン層（ML63Q2557）」、I2C テレメトリを Wi-Fi へ中継する「ゲートウェイ層（ATOMS3 Lite）」、ブラウザで状態と履歴を表示する「ダッシュボード層（plant-medical）」の3層で構成されています。Wi-Fi や PC がつながっていなくても、マイコン基板だけで見守りと給水を続けられます。



主要仕様（ファームウェア実装値）

項目	仕様・設定値
メインMCU	ROHM ML63Q2557（Arm Cortex-M0+、Flash 256 KB、RAM 16 KB、ベアメタル C 実装）
AIアクセラレータ	内蔵ハードウェア `AI_PERI`（Solist-AI™ ライブラリ、3層自己符号化器 8-64-8）
活性化・損失関数	Sigmoid 活性化関数（`ODL_ACTV_SIGMOID`）、二乗平均誤差（`ODL_LOSS_MSE`）、忘却係数 $\lambda = 1.0$（`0x3F80`）
制御・計測周期	システム Tick 10 ms（`PLANT_DOCTOR_TICK_MS`）、センサー取得・AI 更新 1.0 秒（100 ticks）
ストレス算出	0〜100、重みは土壌 35・熱 35・日照 15・湿度 15、最大値支配度 75%
AI異常度判定	健康サンプル 600 件（変化速度履歴 15 分待機後）で基準確立、異常度 50 以上が約 1 分で `WARNING`、30 未満が約 2 分で解除
学習保護・休止	給水後 5400 サンプル（約 1 時間半）は AI 判定・学習を休止。異常時は 60 サンプル（1 分）に 1 回へ間引き、10800 サンプル（3 時間）継続で学び直し
給水制御	1 回最大 10.0 秒（約 20 mL）、クールダウン 3.0 秒、自動給水は最小間隔 30 分・1 日最大 6 回、給水失敗は 30 分で解除
ゲートウェイ通信	I2C アドレス `0x42`、500 ms 周期、40 バイト固定長パケット（16-bit 加算チェックサム付き）

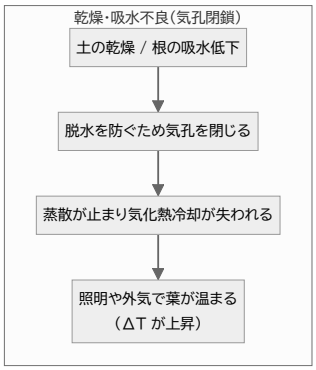
第1章 エッジAIと植物生体センシング技術

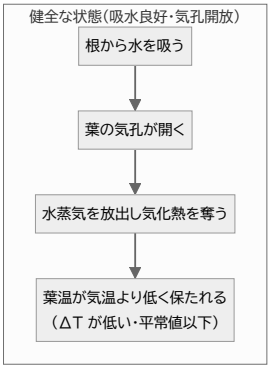
植物は水が足りなくなったり暑すぎたりしても、声を出して知らせられません。葉がしおれたり変色したりして人が気づいたときには、すでに根や葉の組織が傷んでいることも少なくありません。

Plant Doctor は、目に見える変化が出る前の段階で、葉の温度や土の乾き方から植物の体調を読み取り、必要に応じて自動で水やりまで行う装置です。第1章では、植物の蒸散作用を手がかりに生体状態を測る原理と、5種類のセンサーの具体的な計測処理、そしてマイコン基板1枚で判断まで完結させるエッジAI構成について解説します。

1.1 蒸散と気化熱から植物の体調を読む

植物が根から吸い上げた水を水蒸気として葉から放出する働きを**蒸散(Transpiration)**と呼びます。この蒸散が順調に行われているかどうかは、葉の温度と周囲の気温を比べることで外から非接触で確かめられます。





気孔の開閉と葉温の変化

葉の表面にある小さな穴(気孔)は、光合成のために二酸化炭素を取り込むと同時に、内部の水分を蒸発させています。水が液体から気体へ変わるときには周囲から気化熱($20^\circ\mathrm{C}$ で約 $2.45 \times 10^6\ \mathrm{J/kg}$)を奪うため、十分に水を吸って蒸散している葉は周囲の空気より冷やされます。

反対に、土が乾いてきたり根が傷んで水を吸えなくなったりすると、植物はしおれるのを防ぐために気孔を閉じます。すると気化熱による冷却が止まり、照明や日射の熱を受けて葉の温度が上がります。

葉温と気温の差（ΔT）を使う理由

葉の温度（T_{leaf}）だけを測っても、部屋自体が涼しいのか、蒸散によって葉が冷えているのかは区別できません。そこで Plant Doctor では、葉面温度と周囲の気温（T_{air}）の差である**葉気温差** ΔT を主要な指標として使います。

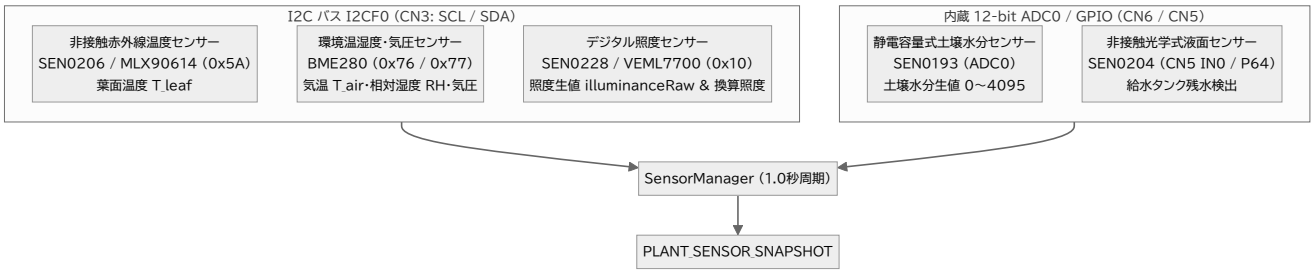
$$\Delta T = T_{\mathrm{leaf}} - T_{\mathrm{air}}$$

ファームウェア（`PlantFeature.c`）では、葉温と気温をそれぞれ $1/100\,^{\circ}\mathrm{C}$（centi-℃）単位の整数で保持し、両方のセンサー値が有効なときに差分 `leafAirTemperatureDelta` を計算します。

- **平常時の目安:** 設定ファイル（`PlantDoctorConfig.h`）では、平常時の基準値 `PLANT_DOCTOR_STRESS_BASELINE_TEMP_DELTA` を $+0.80\,^{\circ}\mathrm{C}$（`80`）としています。照明の熱を受けながらも蒸散が働いていれば、ΔT はマイナスからこの基準値以下に収まります。実機ログ（9月21日〜22日の記録）でも、給水が行われたあとには葉気温差が下がり、蒸散が回復する様子が記録されています。
- **熱ストレス・吸水不良の目安:** ΔT が $+2.00\,^{\circ}\mathrm{C}$（`200`）以上で、かつ葉温の上昇速度が $+0.50\,^{\circ}\mathrm{C/h}$（`50`）以上のときは熱ストレス（`PLANT_STATUS_HEAT_STRESS`）と判定します。また、土が湿っている（$400‰$ 以上）のに ΔT が $+1.50\,^{\circ}\mathrm{C}$（`150`）以上で、給水しても葉が冷えない場合は、根の吸水不良（`PLANT_STATUS_ROOT_UPTAKE`）を疑います。

1.2 5種類のセンサーとファームウェアの計測処理

DT-EBML63Q2557 ボードには、植物と周囲の環境を測る5種類のセンサーを接続しています。`SensorManager` が1秒（100 ticks）ごとにすべてのセンサーを読み取り、最新のスナップショット `PLANT_SENSOR_SNAPSHOT` を更新します。



1. 非接触赤外線温度センサー（SEN0206 / MLX90614）

葉から出る赤外線をサーモパイル素子で受け取り、葉を傷つけずに表面温度を測ります（I2C アドレス `0x5A`）。

- `LeafTemperatureSensor.c` でオブジェクト温度レジスタを読み出し、PEC（CRC-8）とエラービットを検証したうえで $1/100\,^{\circ}\mathrm{C}$ 単位の整数 `leafTemperatureCentiC` に変換します。
- 妥当性判定（`SensorPlausibility.c`）では、$-70.00\,^{\circ}\mathrm{C} \sim +120.00\,^{\circ}\mathrm{C}$ の範囲外や、気温との差 $|\Delta T|$ が $20.00\,^{\circ}\mathrm{C}$ を超える状態が5回続いた場合にセンサー異常と判定します。

2. 環境温湿度・気圧センサー（BME280）

鉢のまわりの気温、相対湿度、気圧を測ります（I2C アドレス `0x76` または `0x77` を自動検出）。

- `EnvironmentSensor.c` は初期化時にチップ ID（`0x60`）を確認し、素子固有のトリミング補正値（`dig_T1` 〜 `T3`、`dig_P1` 〜 `P9`、`dig_H1` 〜 `H6`）を読み出します。浮動小数点を使わずに 32 ビット整数演算で温度補正を行い、気温

`airTemperatureCentiC`（$1/100^\circ\mathrm{C}$ 単位）、相対湿度 `relativeHumidityCentiPercent`（$1/100\%$ 単位、$0 \sim 10000$）、気圧 `barometricPressurePa`（Pa 単位）を算出します。

- 動作中に通信が切れても、次の周期で自動的に再プローブして復帰できる作りになっています。

3. デジタル照度センサー（SEN0228 / VEML7700）と「生値」の扱い

植物に当たる光の強さを測ります（I2C アドレス `0x10`）。ここで特に注意したいのが、**ファームウェア内部とテレメトリにおける「生値（Raw）」と「ルクス（lx）換算値」の使い分け**です。

```
/* sensors/EnvironmentSensor.c: SEN0228 (VEML7700) の読み取りと換算 */
sample->illuminanceRaw = (uint16_t)response[0] | ((uint16_t)response[1] << 8U);
sample->illuminanceCentiLux = (((uint32_t)sample->illuminanceRaw * 576UL) + 50UL) / 100UL;
```

- **換算式:** レジスタ `0x04` から読み出した 16 ビットのカウント値が `illuminanceRaw`（生値）です。ファームウェアでは生値に 5.76 を掛けて $1/100$ lx 単位の `illuminanceCentiLux` を求めており、**生値 1 は約 0.0576 lx** に相当します。実機ログで照明がついている時間帯の生値は約 `1350` であり、ルクスに換算すると約 78 lx です。
- **どこでどちらを使うか:**
- **ローカル LCD（ページ 2）:** `illuminanceCentiLux` を使い、ルクス換算値（`LUX:77.76` など）を表示します。
- **特徴量・AI 入力・日照不足判定・I2C テレメトリ:** すべて**センサーの生値 `illuminanceRaw` をそのまま使います。**具体的には、Solist-AI への第6入力 x_5 は生値 $0 \sim 6000$ を $0 \sim 256$ に正規化し、24時間の積算照度 `illuminanceAccumulated` も1分ごとに生値を足し上げます（日照不足のしきい値 `144000` は、生値 100 × 60分 × 24時間）。また、ATOMS3 Lite へ送るテレメトリパケットの `pkt.illuminanceRaw` も生値です（ダッシュボード側ではこの生値をそのまま表示しています）。

4. 静電容量式土壌水分センサー（SEN0193）と2点校正

土壌の誘電率変化を電圧として出力する腐食しにくいセンサーで、ML63Q2557 内蔵の 12 ビット逐次比較型 ADC（`SA-ADC0`）で読み取ります。

```
/* sensors/SoilMoistureSensor.c: 12-bit ADC 取得と反転 */
adcRaw = (uint16_t)((saAdc0_getResult0() & 0xFFF0UL) >> 4U);
/* CN6の反転増幅回路に合わせ、0が乾燥側、4095が湿潤側となるよう反転する。 */
*rawValue = (uint16_t)(4095U - adcRaw);
```

- **2点校正によるパーミル変換:** 生値 `soilMoistureRaw`（$0 \sim 4095$）は、土壌の種類や挿し方によって幅が変わります。そこで `SoilCalibration_ToPermille` により、乾燥時の校正値 `calDry`（既定値 `1000`）と湿潤時の校正値 `calWet`（既定値 `3000`）の間を $0 \sim 1000$‰（$0.0\% \sim 100.0\%$）へ線形変換します。
- **実行中の校正変更:** UART コンソールから `C=1200,2800` のように送ることで（乾湿の差が `200` 以上あるか検証したうえで）校正値を変更でき、`PlantAi` や `AppStateMachine` の換算にも即座に反映されます。

5. 非接触光学式液面センサー（SEN0204）

給水タンクの外壁に取り付け、プリズムの屈折変化で水の有無を検出します（`CN5 IN0` / `P64`、Low アクティブ入力）。タンクが空（`tankLiquidDetected == false`）のときは自律給水も手動給水もブロックし、ポンプの空運転を防ぎます。

1.3 なぜクラウドではなくボード1枚で動かすのか

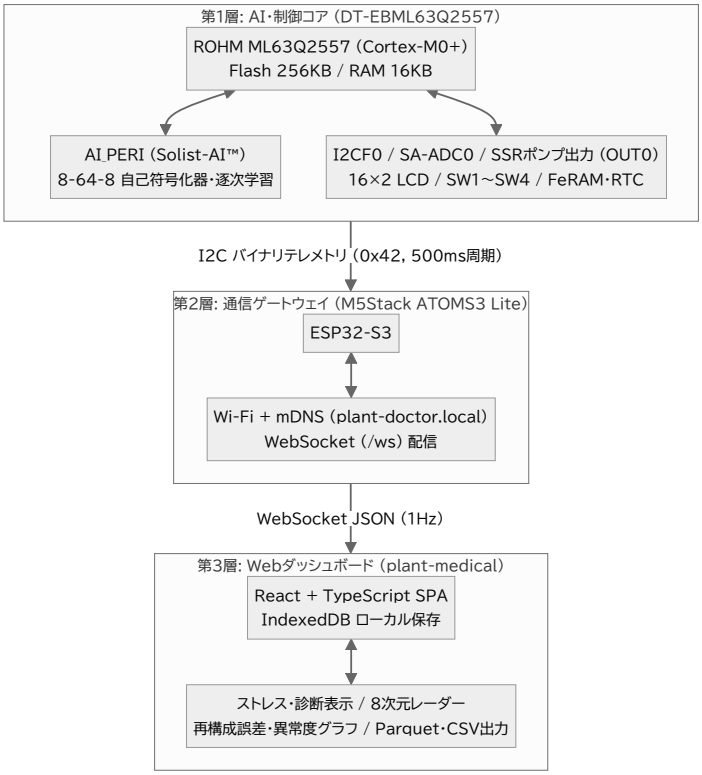
植物の見守りと水やりをクラウドサーバー任せにすると、Wi-Fi が切れたときに水やりが止まったり、逆に停止命令が届かず水があふれたりする危険があります。Plant Doctor は、**センシング・診断・オンデバイス学習・ポンプ制御のすべて**

を DT-EBML63Q2557 ボード1枚で完結させています。

観点	クラウド依存型	Plant Doctor（エッジ完結型）
通信が切れたとき	診断も水やりも止まる	**ML63Q2557 単体でいつもどおり診断・給水を継続**
学習データの扱い	何万件ものデータをサーバーへ送り蓄積する	**その場で1サンプルずつ重みを更新し、データは即破棄**
ポンプの安全停止	通信遅延の影響を受ける	**10 ms 周期のメインループ内で最大10秒制限・緊急遮断**
設置場所の自由度	常時ネット回線が必要	**ネットのない温室や展示会場でもそのまま動く**

1.4 3層ハイブリッド構成の役割分担

エッジ単体で完結する安全性を土台にしつつ、離れた場所からの観察や記録の分析もしやすいよう、システム全体を3つの層に分けています。



1. **第1層：ROHM ML63Q2557（DT-EBML63Q2557）** RTOS を使わない 10 ms Tick のスーパーループで動作します。1秒ごとにセンサーを読み取り、ルール診断とストレス算出、Solist-AI（`AI_PERI`）による「いつもと違う」判定と学習、SSR 出力（`OUT0` / `P66`）によるポンプ駆動（最大10秒＝約20 mL）、16×2 LCD の表示更新、FeRAM へのログ記録を行います。
2. **第2層：M5Stack ATOMS3 Lite（ESP32-S3）** 表示・通信用のゲートウェイです。ML63Q2557 から I2C（アドレス `0x42`）で 500 ms ごとに 40 バイトのテレメトリを受け取り、Wi-Fi の WebSocket（`/ws`）でブラウザへ配信します。また、ブラウザからの給水指示を I2C 経由で ML63Q2557 へ伝えます。Wi-Fi 処理はすべてこちらで引き受けるため、通信の負荷が ML63Q2557 の制御周期に影響しません。

3. **第3層：Web ダッシュボード（plant-medical）** PC やスマートフォンのブラウザで開く観察画面です。受信したデータをブラウザ内の IndexedDB に保存し、ストレススコア、8次元特徴量、Solist-AI の再構成誤差と異常度、給水履歴をグラフで表示します。

次の第2章では、まずルールベースで植物の危機を見逃さず安全に水をやるための「ストレスバロメータ」と「自律診断・給水アルゴリズム」を詳しく見ていきます。

第2章 多変量植物ストレス評価と自律診断アルゴリズム

植物の状態を「土が乾いているか・湿っているか」の二択だけで判断すると、強い照明による葉の過熱や、給水しても根が水を吸えていない異常を見落としてしまいます。かといって、すべてを機械学習だけに任せてしまうと、水やりの開始や停止の理由がブラックボックスになり、安全な制御が難しくなります。

そこで Plant Doctor は、植物生理学にもとづくルールベースの評価と、第3章・第4章で解説する Solist-AI の学習を組み合わせています。本章では、4つの環境・生体指標から 0〜100 の数値を算出する**ストレスバロメータ**（`PlantStress.c`）、優先順位にもとづいて原因候補を特定する**状態診断**（`PlantDiagnosis.c`）、そして安全条件と事後確認を備えた**自律給水**（`WateringPolicy.c`・`WateringResponse.c`）の仕組みを解説します。

2.1 ストレスバロメータ（0〜100）の役割と全体像

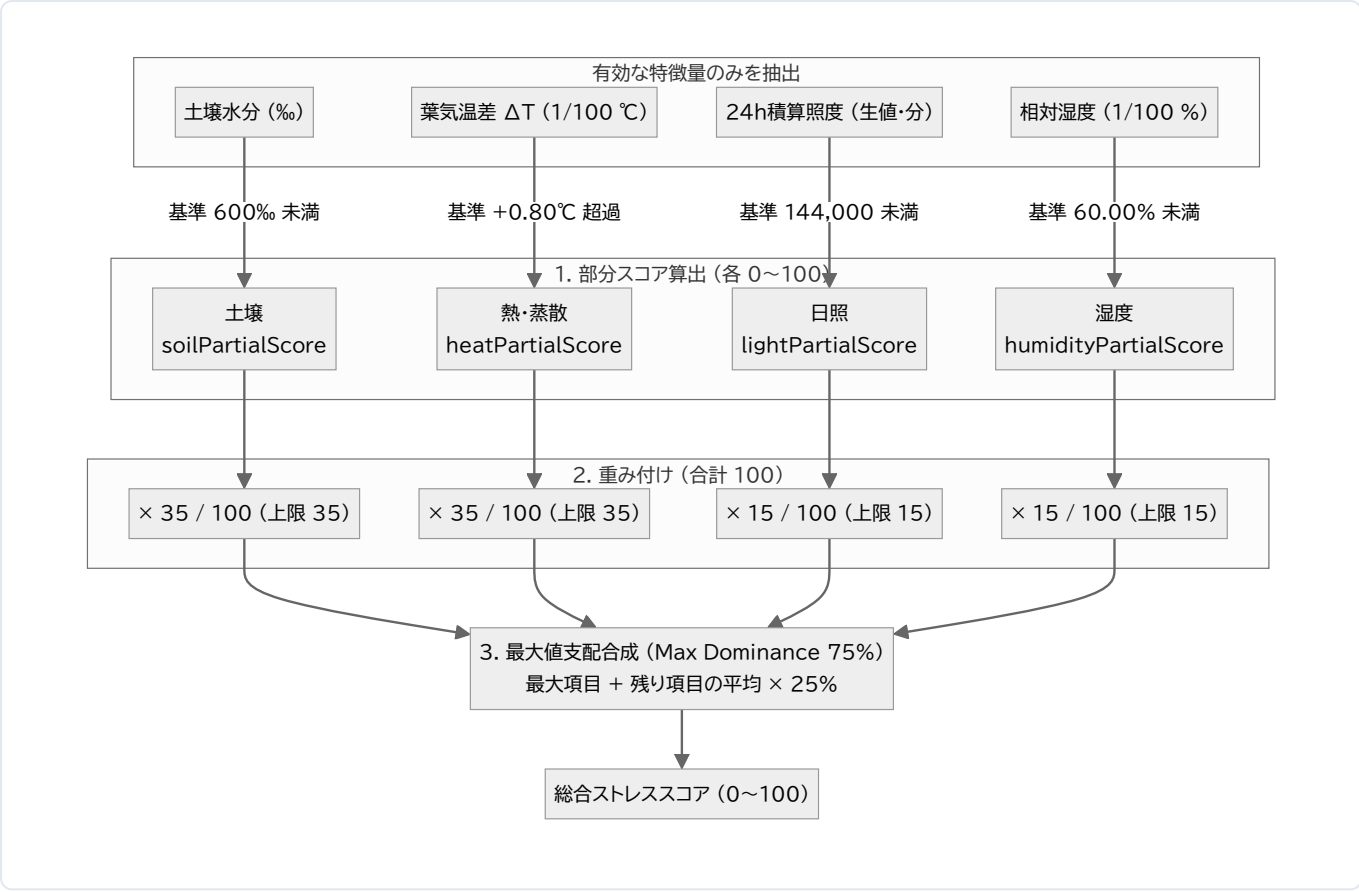
ストレスバロメータは、植物が受けている負荷の大きさを 0〜100 の連続した数値で表す指標です。平常時のベーススコア `PLANT_DOCTOR_STRESS_BASE_SCORE` は `0` に設定されており、すべての指標が基準範囲内にあればスコアは **0** になります。

このスコアは LCD や Web ダッシュボードに表示されるだけでなく、**Solist-AI に現在のデータを「いつもの健康な状態」として学習させてよいかどうかの関門**としても使われます。

スコアの目安	植物と環境の状態	システムの主な動き
0 〜 29	土壌水分・葉気温差・日照・湿度がいずれも良好（または軽微な変化）	ルール診断が `HEALTHY` などの条件を満たせば **Solist-AI が毎秒学習を実行**
30 〜 49	土壌の乾き始めや空気の乾燥など、中程度の負荷	**定常時の AI 学習を停止**し、異常データを「いつも」として覚えるのを防ぐ（立ち上げの100回未満のみ、土壌・熱の部分スコア40未満なら50未満まで学習許可）
50 以上	乾燥や熱ストレスなど、複数の負荷または強い負荷が重なった状態	AI 学習を完全に止め、ルール診断に応じた警告表示や自動給水を行う

2.2 4項目の負荷指標（0〜100）の計算式

`PlantStress_Evaluate`（`ai/PlantStress.c`）では、まず通信状態と妥当性チェック（`SENSOR_HEALTH_OK`）を通過した項目だけを対象に、4つの**部分スコア（Partial Score、各 0〜100）**を求めます。異常と出たセンサーや、まだ履歴がそろっていない項目は対象から外し（マスク）、0点で全体を薄めてしまわないよう配慮しています。



1. 土壌の渇き(`soilPartialScore`)

根の周りの水分が平常基準 `baselineSoilMoisturePermille = 600` ($600‰ = 60.0\%$)をどれだけ下回ったかを 0〜100 に換算します。

$$\text{soilPartialScore} = \min\left(100,\ \left\lfloor \frac{\max(0,\ 600 - \text{soilMoisturePermille}) \times 100}{600} \right\rfloor\right)$$

- $600‰$ 以上なら 0、給水開始ラインの $300‰(30.0\%)$で 50、完全なカラカラ状態($0‰$)で 100 になります。

2. 葉のほてり·蒸散低下(`heatPartialScore`)

葉気温差 $\Delta T = T_{\text{leaf}} - T_{\text{air}}$ が平常基準 `baselineTempDeltaCentiC = 80` ($+0.80^\circ\text{C}$)を超えた分を、フルスケール幅 `maxHeatDeltaRangeCentiC = 167` ($+1.67^\circ\text{C}$)で割って 0〜100 にします。

```
/* ai/PlantStress.c: 熱ストレス部分スコアの算出 */
if ((config->maxHeatDeltaRangeCentiC > 0) &&
    (features->leafAirTemperatureDelta > config->baselineTempDeltaCentiC)) {
    int32_t diff = features->leafAirTemperatureDelta - config->baselineTempDeltaCentiC;
    int32_t score = (diff * 100L) / config->maxHeatDeltaRangeCentiC;
    if (score > 100L) {
        score = 100L;
    }
    output->heatPartialScore = (uint8_t)score;
}
```

- $\Delta T \leq +0.80^\circ\text{C}$ のときは 0、気孔が閉じて $\Delta T \geq +2.47^\circ\text{C}(0.80 + 1.67)$に達すると上限の 100 になります。

3. 光量不足（`lightPartialScore`）

直近24時間の積算照度 `illuminanceAccumulated`（1分ごとに照度センサーの生値 `illuminanceRaw` を足し上げた24時間合計）を見ます。起動から丸1日分の履歴がたまるまでは評価から除外されます。

- 下限目安 `baselineIlluminanceAccum = 144000`（生値の平均 100×60分 $\times$ 24時間、ルクス換算で平均約 5.8 lx 相当）を割り込むと、不足割合に応じて $0 \sim 100$ がつきます。

4. 空調などによる空気の乾き（`humidityPartialScore`）

相対湿度が平常基準 `baselineHumidityCentiPercent = 6000`（$60.00\%\mathrm{RH}$）を下回った割合です。たとえば湿度 30.00% なら 50 になります。

2.3 重み(35 / 35 / 15 / 15)と最大値支配合成

4つの指標を1つに束ねる際、`PlantDoctorConfig.h` ではそれぞれの重みを次のように定めています。

```
/* config/PlantDoctorConfig.h */
#define PLANT_DOCTOR_STRESS_WEIGHT_SOIL          (35U)  /* 土壌水分の重み */
#define PLANT_DOCTOR_STRESS_WEIGHT_HEAT          (35U)  /* 熱ストレスの重み */
#define PLANT_DOCTOR_STRESS_WEIGHT_LIGHT         (15U)  /* 日照の重み */
#define PLANT_DOCTOR_STRESS_WEIGHT_HUMIDITY      (15U)  /* 湿度の重み */
#define PLANT_DOCTOR_STRESS_MAX_DOMINANCE_WEIGHT (75U)  /* 最大値支配度[%] */
```

重みを 35 / 35 / 15 / 15 にしている理由

各指標 $S_i \in [0, 100]$ には、まず重み w_i が掛けられます。

$$W_i = \left\lfloor \frac{S_i \times w_i}{100} \right\rfloor$$

枯死に直結する**土壌の渇き(35)**と**葉のほてり(35)**を主役に据え、穏やかに効く**光量(15)**と**空気の乾き(15)**を脇役に置いて、4つの重みの合計がちょうど $35 + 35 + 15 + 15 = 100$ になるよう配分しています。

以前は `PlantAi_Init` 内に `100 / 100 / 80 / 60` という大きな重みが直接書かれており、設定ファイルの配分が効いていませんでした。しかし重みが 100 や 60 だと、土の水分が 42%（420‰、指標値 30）まで減ったり、部屋の湿度が 30%（指標値 50、重み付き 30）になったりしただけで総合値が 30 に達し、植物は元気（`HEALTHY`）なのに Solist-AI の学習が止まってしまう弊害がありました。さらに実機ログ（`0921.json`・`0922.json`）の記録値と比べても、`35 / 35 / 15 / 15` のほうが実機の出力にずっと近く（平均差 4.9、旧設定では 17.8）、日常の緩やかな乾きの範囲では総合値が 30 未満に収まって AI が順調に学び続けられます。

最大値支配(Max Dominance)による合成式

有効な項目の重み付き値 W_i のうち、最も大きいものを $P_{\max}$、残りの合計を P_{other}、有効項目数を N とすると、総合値は次式で決まります（`baseStressScore = 0` のとき）。

$$\text{stressScore} = \min\left(100,\ P_{\max} + \left\lfloor \frac{P_{\text{other}} \times (100 - 75)}{100 \times (N - 1)} \right\rfloor\right)$$

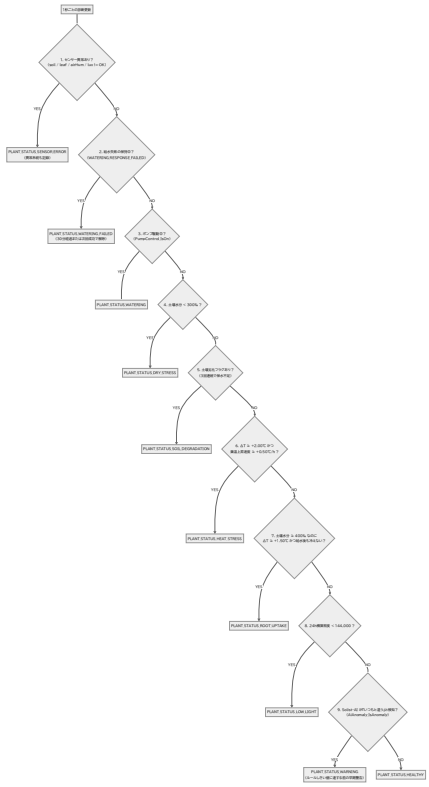
```c
/* ai/PlantStress.c: 最大値支配による合成 */
secondaryFactor = 100 - (int32_t)config->maxDominanceWeight; /* 25 */
secondaryAdd = 0;
if (count > 1) {
    secondaryAdd = (otherWeightedSum * secondaryFactor) / (100 * (count - 1));
}
deviation = maxWeightedPartial + secondaryAdd;
if (deviation > 100) {
    deviation = 100;
}
```

全項目を単純平均すると、1つの要素だけが跳ね上がっても残りのゼロで割られて4分の1に薄まってしまいます。最大値支配方式なら、最大の重み付き値 $P_{\max}$ をそのまま土台にし、他の要素も重なったときだけその平均の 25% を上乗せできます。

2.4 優先順位付きの原因候補診断と AI `WARNING` の統合

ストレスバロメータが「どれくらい負担がかかっているか」という量を表すのに対し、「いま何が起きているか」という原因候補を絞り込むのが `PlantDiagnosis_Update`（`ai/PlantDiagnosis.c`）と `PlantAi.c` です。

次の優先順で上から順に照合し、最初に当てはまったものを現在のステータス（`PLANT_STATUS`）とします。



ルール診断と Solist-AI 判定の役割分担

1〜8番目までは、誰の目にも明らかな危険や機器トラブルをルールで確実に押さえます。そして1〜8番目のどれにも該当せず、ルール上は `PLANT_STATUS_HEALTHY` と出たときに初めて、9番目の **Solist-AI による「いつもと違う」検知**（**`AiAnomaly_IsAnomaly`**）を確認します。

```
/* ai/PlantAi.c: ルール上は健康でも、AIが「いつもと違う」と判定したら WARNING にする */
if ((s_diagnosisState.status == PLANT_STATUS_HEALTHY) && AiAnomaly_IsAnomaly(&s_aiAnomaly)) {
    s_diagnosisState.status = PLANT_STATUS_WARNING;
}
```

たとえば土の水分がまだ $460‰$（46%）残っていて乾燥しきい値（$300‰$）には届いていなくても、「普段より土の減り方が速く、葉温の上がり方も違う」と Solist-AI が察知すれば、この時点で `PLANT_STATUS_WARNING` に切り替えて画面やダッシュボードへ知らせます。

2.5 自律給水の8条件と給水後の自己診断

植物が乾燥ストレス（`PLANT_STATUS_DRY_STRESS`）になったとき、マイコンはポンプ（流量約 $120\ \mathrm{mL/min}$）を最大 **10.0秒間（`PLANT_DOCTOR_PUMP_MAX_ON_TICKS = 1000`、約 20 mL）** 動かして水をやります。ただし、水浸しや空運転を防ぐため、`WateringPolicy_Evaluate`（`app/WateringPolicy.c`）で次の8つの条件をすべて満たしたときだけ自動給水を実行します。

1. 診断ステータスが `PLANT_STATUS_DRY_STRESS` であること。
2. 土壌水分の移動平均が `300‰`（30.0%）未満であること。
3. 自動給水フラグ `autoWateringEnabled` が有効（`true`）であること。
4. 土壌水分センサーが正常（`SENSOR_HEALTH_OK`）であること。
5. タンクに水がある（`tankLiquidDetected == true`）こと。
6. 前回の給水から **30分（`1800` 秒 / `180000` ticks）以上** あいていること。
7. 直前の給水応答が失敗（`WATERING_RESPONSE_FAILED`）でないこと。
8. 当日の給水回数が **6回（`maxDailyCount = 6`）未満** であり、土壌劣化（`SOIL_DEGRADATION`）でもないこと。

なお、手動給水（SW4・コンソールの `W`・ダッシュボードの給水ボタン）を行った場合も、水のやりすぎを防ぐために給水時刻と回数が記録され、そのあと30分間は自動給水を控える設計になっています。

給水後の自己診断（30秒・3分・5分）と30分での失敗解除

水をやったあと、`WateringResponse_Update`（`ai/WateringResponse.c`）が経過時間（`s_aiTick` 基準）を追跡し、水がきちんと土と植物に届いたかを3段階で確かめます。

判定タイミング	設定マクロ（既定値）	確認内容と判定結果
1. 直後確認	`PLANT_DOCTOR_RESP_IMMEDIATE_SECONDS`（**30秒**）	土壌水分が給水前より $+50‰$（**5ポイント**）**以上** 増えたか。増えていれば浸透確認へ進む
2. 浸透確認	`PLANT_DOCTOR_RESP_DELAYED_SECONDS`（**180秒＝3分**）	30秒時点で未達でも、3分後に $+50‰$ **以上** 増えていれば土壌回復 OK。3分たっても $+50‰$ 未満なら、チューブ外れや水量不足とみなして **`WATERING_RESPONSE_FAILED`（給水失敗）** にする
3. 蒸散回復確認	`PLANT_DOCTOR_RESP_TEMP_DELTA_SECONDS`（**300秒＝5分**）	土壌水分が回復したあと、5分後に葉気温差 ΔT が給水前より $-0.30^{\circ}\mathrm{C}$（**`30`**）**以上** 下がったか。下がらなければ `wateringDidNotCoolLeaf = true` とし、吸水不良（`ROOT_UPTAKE`）の判断材料にする

また、給水による土壌水分の増加量が $+80‰$ 未満しか保てない状態が3回連続した場合は、土の保水力が落ちたとして土壌劣化（`isSoilDegraded = true`）と判定します。

```
/* ai/WateringResponse.c: 給水失敗は一定時間(30分)が過ぎたら解除し、再び自動給水を試せるようにする */
if ((state->result == WATERING_RESPONSE_FAILED) && (config->failedHoldTicks > 0UL)) {
    if ((input->currentTick - state->wateringStartTick) >= config->failedHoldTicks) {
        state->result = WATERING_RESPONSE_NONE;
    }
    return;
}
```

一度 `WATERING_RESPONSE_FAILED` になると、優先順位とインターロックによって自動給水が止まります。以前はこれを発報後に解除する処理がなかったため、たとえば SW4 でポンプを動かしてすぐ止めたときや、もともと湿り気のある土に手動で足し水をして $+50‰$ 増えなかったときに、電源を入れ直すまで自動給水が止まったままになっていました。現在のファームウェアでは `PLANT_DOCTOR_RESP_FAILED_HOLD_SECONDS = 1800UL`（30分）が経過すると自動的に `WATERING_RESPONSE_NONE` へ戻り、再び自動給水を試せるようになっています。

さらに、SW4 だけでなく UART コンソールの `W` や Web ダッシュボードからの給水指示もすべて `AppStateMachine_RequestManualWatering` に統一し、給水前の土壌水分と葉気温差を必ず `PlantAi_NotifyWatering` に記録してからポンプを動かすことで、どの経路で給水しても上記の自己診断が正しく働くようになっています。

第3章 自己符号化器(Autoencoder)と多次元生体異常検知

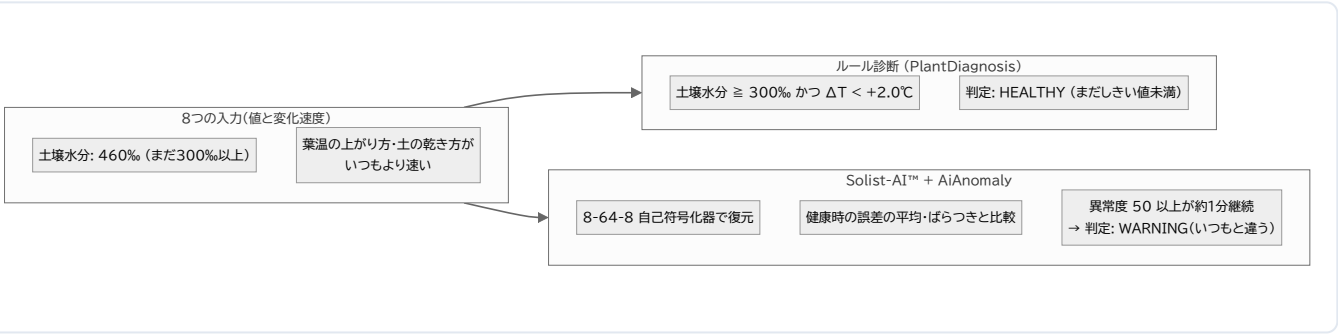
第2章で見たルールベースの診断は、「土壌水分が $300‰(30\%)$ を切ったら水やり」「葉気温差 ΔT が $+2.00^\circ\mathrm{C}$ 以上なら熱ストレス」といった明確な危険を確実に捉えます。しかし、固定のしきい値だけでは、「普段は1時間に 0.6 ポイントずつゆっくり乾く鉢なのに、今日は1時間に 6 ポイントの速さで乾き始め、葉温もじわじわ上がっている」といった、**しきい値に達する前の異変**には気づけません。

また、植物の種類や鉢の大きさ、土の配合、照明との距離が違えば、「普段の乾き方」や「照明がついたときの葉温の上がり方」は鉢ごとに異なります。本章では、ROHM ML63Q2557 に内蔵された **Solist-AI™** の自己符号化器(Autoencoder)を使ってその株の「いつも」を学び、再構成誤差のずれから異常度(0〜100)と「いつもと違う」(`WARNING`)を判定する仕組み(`PlantAi.c`・`AiAnomaly.c`)を詳しく解説します。

3.1 なぜ「元気なときの姿だけ」を学ぶのか

植物の不調には、水切れ、根詰まり、根腐れ、日照の変化、空調の直撃など多様な要因があり、あらかじめ「病気のときのデータ」を網羅して覚えさせるわけにはいきません。一方で、目の前にある鉢植えから毎日届く記録の大部分は「その株の普段どおりの姿」です。

そこで Plant Doctor では、**健康なときのセンサーの組み合わせだけを自己符号化器に記憶させ、普段の調和から外れたときに復元ロス(再構成誤差)が膨らむ性質を使った教師なし異常検知**を行っています。



3.2 Solist-AI に入れる8次元の特徴量と正規化

`PlantAi_Process10Ms`(`ai/PlantAi.c`)では、1秒ごとに取得したセンサー値とその変化速度から **8次元の特徴量ベクトル** $\mathbf{x} = [x_0, x_1, \ldots, x_7]^T$ を作り、Solist-AI に入力します。

8つの入力項目と正規化範囲

Solist-AI の出力層は Sigmoid 活性化関数(`ODL_ACTV_SIGMOID`、出力範囲 $0.0 \sim 1.0$)を使うため、入力値も $[0.0, 1.0]$ に収める必要があります。ファームウェアでは `NormalizeToQ8` 関数で各物理量を $0 \sim 256$(固定小数点 Q8 形式の $0.0 \sim 1.0$)へ線形変換したあと、`ODL_ToBfloat16` で 16 ビット浮動小数点(`bfloat16`)の配列に変換します。

要素	特徴量(変数名)	正規化範囲(最小〜最大)	単位・設定の意図
x_0	**土壌水分**(`soilMoisturePermille`)	$0 \sim 1000$	‰($0.0\% \sim 100.0\%$)。校正後の土壌水分
x_1	**葉面温度**(`leafTemperatureCentiC`)	$1000 \sim 4000$	$1/100^\circ\mathrm{C}$($10.00^\circ\mathrm{C} \sim 40.00^\circ\mathrm{C}$)
x_2	**環境気温**(`airTemperatureCentiC`)	$1000 \sim 4000$	$1/100^\circ\mathrm{C}$($10.00^\circ\mathrm{C} \sim 40.00^\circ\mathrm{C}$)
x_3	**葉気温差** ΔT(`leafAirTemperatureDelta`)	$-400 \sim +200$	$1/100^\circ\mathrm{C}$($-4.00^\circ\mathrm{C} \sim +2.00^\circ\mathrm{C}$)
x_4	**相対湿度**(`relativeHumidityCentiPercent`)	$2000 \sim 10000$	$1/100\%$($20.00\% \sim 100.00\%\mathrm{RH}$)
x_5	**照度(生値)**(`illuminanceRaw`)	$0 \sim 6000$	**照度センサーの生値**(1カウント ≈ 0.0576 lx、点灯時は約1350)
x_6	**葉温の変化速度** (`leafTemperatureRatePerHour`)	$-300 \sim +300$	$1/100^\circ\mathrm{C/h}$($-3.00 \sim +3.00^\circ\mathrm{C/h}$)
x_7	**土壌水分の変化速度** (`soilMoistureRatePerHour`)	$-60 \sim +60$	‰/h($-60 \sim +60$‰/h、すなわち $\pm 6.0\%/\mathrm{h}$)

```
/* ai/PlantAi.c: 8次元特徴量の Q8 正規化 (0〜256) */
bool isRateReady = (s_featureState.minuteHistoryCount >= PLANT_DOCTOR_AI_RATE_READY_MINUTES);
rawFeatures[0] = NormalizeToQ8(featureInput.soilMoisturePermille, 0, 1000);
rawFeatures[1] = NormalizeToQ8(snapshot.leafTemperatureCentiC, 1000, 4000);
rawFeatures[2] = NormalizeToQ8(snapshot.airTemperatureCentiC, 1000, 4000);
rawFeatures[3] = NormalizeToQ8(s_featureVector.leafAirTemperatureDelta, -400, 200);
rawFeatures[4] = NormalizeToQ8(snapshot.relativeHumidityCentiPercent, 2000, 10000);
rawFeatures[5] = NormalizeToQ8(snapshot.illuminanceRaw, 0, 6000);
/* 変化速度は範囲を狭め、その株らしい葉温の上がり方・土の乾き方からのずれを誤差に表れやすくする */
rawFeatures[6] = isRateReady ? NormalizeToQ8(s_featureVector.leafTemperatureRatePerHour,
                                             -PLANT_DOCTOR_AI_LEAF_RATE_RANGE_CENTIC,
                                             PLANT_DOCTOR_AI_LEAF_RATE_RANGE_CENTIC) : 128;
rawFeatures[7] = isRateReady ? NormalizeToQ8(s_featureVector.soilMoistureRatePerHour,
                                             -PLANT_DOCTOR_AI_SOIL_RATE_RANGE_PERMILLE,
                                             PLANT_DOCTOR_AI_SOIL_RATE_RANGE_PERMILLE) : 128;
```

変化速度(x_6, x_7)の範囲設定と起動後15分間の待機

このコードには、実機とシミュレーションの検証から生まれた2つの重要な工夫が入っています。

1. **変化速度の正規化範囲を狭めて感度を上げる:** 以前は葉温の変化速度を $\pm 5.00^\circ\mathrm{C/h}$、土壌水分の変化速度を ± 200‰/h で正規化していました。しかしその広い範囲では、普段のゆっくりした乾き(たとえば $0.6\%/\mathrm{h} = 6$‰/h)と異常な速さの乾き($6\%/\mathrm{h} = 60$‰/h)の差が $0 \sim 256$ のスケール上で小さくなり、再構成誤差に表れにくくなります。そこで `PLANT_DOCTOR_AI_LEAF_RATE_RANGE_CENTIC = 300` ($\pm 3.00^\circ\mathrm{C/h}$)、`PLANT_DOCTOR_AI_SOIL_RATE_RANGE_PERMILLE = 60` ($\pm 60$‰/h)へと範囲を絞り、変化のしかたの違いが誤差に大きく反映されるようにしています。
2. **履歴が15分たまるまでは中間値(`128`)を入れる:** `PlantFeature.c` は1分ごとの履歴(最大60件＝60分)の最も古い点と新しい点の差から1時間あたりの変化速度を計算します。そのため、電源を入れた直後で履歴が2〜3分しかないときには、土壌水分がわずか 1‰ 揺れただけで「1時間あたり $20 \sim 30$‰/h」と大きく拡大されてしまい、その時期に作られた誤差の基準のばらつきが大きくなって感度が落ちていました。そこで、1分ごとの履歴が **15件** (`PLANT_DOCTOR_AI_RATE_READY_MINUTES = 15` 、**約15分**) たまるまでは、`isRateReady` を `false` とし、x_6 と x_7 には「変化なし(0)」にあたる中間値 `128` (0.5)を入れます。

3.3 8-64-8 自己符号化器と再構成誤差(ppm)

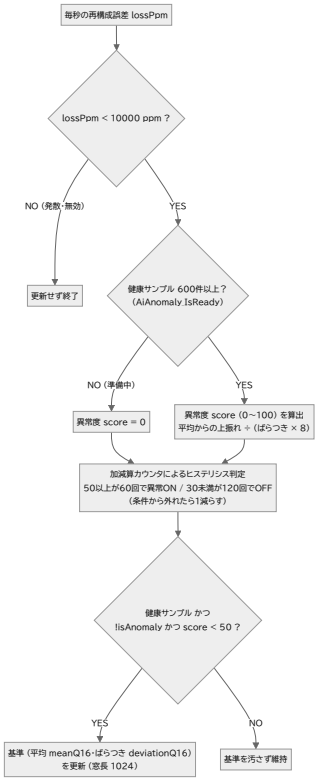
Solist-AI アクセラレータ(`AI_PERI`)は、入力層 8 ノード・隠れ層 64 ノード・出力層 8 ノードの3層自己符号化器を実行します。入力ベクトル $\mathbf{x} \in [0.0, 1.0]^8$ を隠れ層へ圧縮して再び8次元ベクトル $\hat{\mathbf{x}}$ へ復元し、両者の二乗平均誤差(MSE)を `bfloat16` 形式で返します。

$$\mathrm{MSE} = \frac{1}{8}\sum_{i=0}^{7}(x_i - \hat{x}_i)^2$$

ファームウェアでは `OSUAD_GetLoss()` から得た誤差を `0.0f` ～ `1.0f` にクリップしたうえで $10,000$ 倍し、整数 `s_solistAiLatestLossPpm`($0 \sim 10000$ ppm)に変換します。たとえば実機ログ(9月24日 13:00～9月25日 22:20 の健康な状態)における再構成誤差の平均は約 0.0235(ばらつき約 0.0007)であり、ppm 表記では **約 235 ppm(ばらつき約 7 ppm)** にあたります。

3.4 株ごとの誤差基準の学習と異常度(0～100)の算出(`AiAnomaly.c`)

再構成誤差の絶対値は、鉢の環境やセンサーの揺らぎ方によって少しずつ異なります。そのため「誤差が何 ppm を超えたら異常」と固定値で決めるのではなく、`AiAnomaly.c` で**その株が健康なときの再構成誤差の平均(水準)とばらつき(平均絶対偏差)を学び、そこからの上振れを異常度 0～100 に換算**しています。



1. 健康時の水準と揺れ幅の学習

`AiAnomaly_Update` は、次の条件をすべて満たすデータだけを「基準の算出に使ってよい健全な標本」として受け入れます。

- ルール診断が健康かつ低負荷などの関門（`isRuleSafe`）を通過していること
- 起動後15分が経過して推移履歴がそろっていること（`isRateReady == true`）
- Solist-AI の累積学習回数が **100回以上**（`PLANT_DOCTOR_AI_ANOMALY_MIN_TRAIN_COUNT = 100`）に達していること
- 現在「いつもと違う」と出ておらず（`!state->isAnomaly`）、スコアも開始ライン未満（`state->score < 50`）であること

マイコン上で浮動小数点を使わずに済むよう、ppm 値を 16 ビット左シフトした Q16 固定小数点（`meanQ16`・`deviationQ16`）で保持します。最初の 1024 件未満（$n < 2^{10}$）は立ち上がりを速めるため単純平均（$1/n$）で馴染ませ、それ以降は窓長 $2^{10} = 1024$ 件（`PLANT_DOCTOR_AI_ANOMALY_BASELINE_SHIFT = 10`、約17分）の指数移動平均で緩やかに追従させます。

```
/* ai/AiAnomaly.c: 健康時の平均とばらつき（平均絶対偏差）の更新 */
int32_t err = xq - state->meanQ16;
int32_t absErr = (err < 0) ? -err : err;
uint32_t n = state->baselineSamples + 1U;
uint32_t window = 1UL << config->baselineShift; /* 1024 */
if (n < window) {
    /* 立ち上がりは単純平均で早く収束させる */
    state->meanQ16 += err / (int32_t)n;
    state->deviationQ16 += (absErr - state->deviationQ16) / (int32_t)n;
} else {
    state->meanQ16 += err / (int32_t)window;
    state->deviationQ16 += (absErr - state->deviationQ16) / (int32_t)window;
}
```

2. 監視開始までの時間と異常度（0〜100）の計算式

健全な標本が **600 件**（`PLANT_DOCTOR_AI_ANOMALY_MIN_BASELINE_SAMPLES = 600`、**約 10 分**）集まると `AiAnomaly_IsReady` が `true` になり、異常度の算出が始まります。変化速度の履歴がたまるまでの15分と合わせると、**電源投入から約25分で「いつもと違う」の見守りが始まる**計算です（それまでの準備中は `0` を返します）。

準備完了後の `score`（$0 \sim 100$）は、足元のロスが平常水準 `meanQ16` からどれだけ上振れしたかを、普段の揺れ幅 `deviation` の倍率で表した値です。

$$\text{score} = \min\left(100,\ \left\lfloor \frac{\max(0,\ \text{lossPpm} - \text{mean}) \times 100}{\max(\text{deviation},\ 15\ \text{ppm}) \times 8} \right\rfloor\right)$$

- **揺れ幅の下限**（`minDeviationPpm = 15`）: 室温や光が一定でロスがほとんど動かない時間帯に、分母が小さくなりすぎて過敏に反応するのを防ぐため、下限を 15 ppm（MSE で 0.0015）に置いています。
- **倍率の換算**（`deviationScale = 8`）: 平常水準からの上振れが揺れ幅の **8倍** に達したときに **100、4倍** で開始ラインの **50** になります。たとえば水準が 235 ppm、揺れ幅が下限の 15 ppm の株なら、ロスが $235 + 15 \times 4 = 295$ ppm へ跳ね上がると 50 に達します。
- **実機記録での裏付け**: 9月24日〜25日の実機ログ（元気な約21時間）の推移にこの式を当てはめると、最高でも **20** に収まり、警戒ラインの 50 を超えた時点は一度もありませんでした。

3.5 外れたら1つ減らす加減算カウンタとヒステリシス

復元ロスは毎秒わずかに上下するため、たった1秒だけ 50 を超えたからといって即座に `WARNING` を出すと誤報になります。かといって、「60回連続で 50 以上なら発報し、途中で1度でも 50 を割ったらゼロから数え直す」という単純な連

続方式にすると、50 前後で小刻みに揺れている局面で何度も振り出しに戻され、想定値の試算でも最初に 50 を超えてから `WARNING` が出るまで 44 分を要していました。

そこで `AiAnomaly.c` では、**条件を満たせば 1 増やし、外れてもゼロには戻さず 1 だけ減らす加減算カウンタ(ヒステリシス付き)** を取り入れています。

```
/* ai/AiAnomaly.c: 1回外れても数え直さず、1つ減らすヒステリシス判定 */
if (state->score >= config->onScore) {          /* onScore = 50 */
    if (state->aboveCount < config->onSamples) { /* onSamples = 60 (約1分) */
        ++state->aboveCount;
    }
    if (state->belowCount > 0U) {
        --state->belowCount;
    }
} else if (state->score < config->offScore) {   /* offScore = 30 */
    if (state->belowCount < config->offSamples) {/* offSamples = 120 (約2分) */
        ++state->belowCount;
    }
    if (state->aboveCount > 0U) {
        --state->aboveCount;
    }
} else {                                        /* 30 <= score < 50 */
    if (state->aboveCount > 0U) {
        --state->aboveCount;
    }
    if (state->belowCount > 0U) {
        --state->belowCount;
    }
}
if (!state->isAnomaly && (state->aboveCount >= config->onSamples)) {
    state->isAnomaly = true;
    state->belowCount = 0U;
} else if (state->isAnomaly && (state->belowCount >= config->offSamples)) {
    state->isAnomaly = false;
    state->aboveCount = 0U;
}
```

設定項目	マクロ名	設定値	意味
開始スコア	`PLANT_DOCTOR_AI_ANOMALY_ON_SCORE`	**50**	この値以上で `aboveCount` を +1 する
開始サンプル数	`PLANT_DOCTOR_AI_ANOMALY_ON_SAMPLES`	**60**(約1分)	`aboveCount` が 60 に達したら `isAnomaly = true`(`WARNING`)にする
解除スコア	`PLANT_DOCTOR_AI_ANOMALY_OFF_SCORE`	**30**	この値未満で `belowCount` を +1 する
解除サンプル数	`PLANT_DOCTOR_AI_ANOMALY_OFF_SAMPLES`	**120**(約2分)	`belowCount` が 120 に達したら `isAnomaly = false` に戻す

カウンタの上限をそれぞれ `60` と `120` で頭打ちにしているため、平常状態が何時間続いたあとでも、異変が起きれば最短約1分で `WARNING` になり、収まれば最短約2分で解除されます。

さらに、水やりが行われた直後は土壌水分の変化速度が大きく振れるため、`AiAnomaly_ClearJudgement` によって**学習済みの基準(`meanQ16`・`deviationQ16`・`baselineSamples`)はそのまま残し、判定状態(`score`・`isAnomaly`・カウンタ)だけをリセットして1時間半休止**します。この給水後の休止と、健康なモデルを汚さないための7つの学習関門については、次の第4章で詳しく解説します。

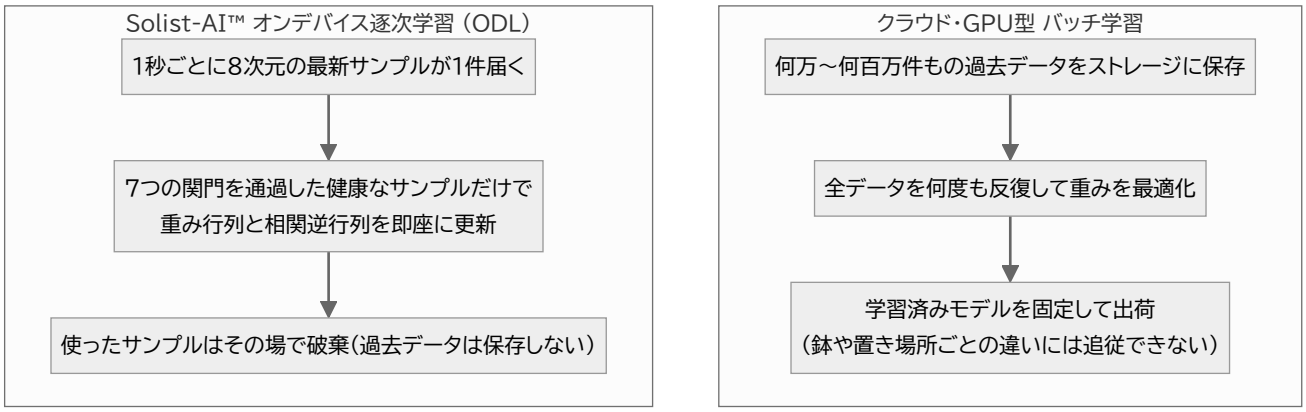
第4章 オンデバイス逐次学習(ODL)の数学と7つの関門

一般的な機械学習では、大量のデータをサーバーに集めて何度も繰り返し学習させる「バッチ学習」が使われます。しかしROHM ML63Q2557 の内蔵 RAM は **16 KB**(Flash は 256 KB)しかなく、1秒ごとに届くセンサーの履歴を何日分も蓄えておく余裕はありません。さらに、植物を現場で見守るエッジ AI にとって最も危険なのは、**「水切れや高温で苦しんでいるときの異常なデータを、いつもの健康な状態として覚えてしまうこと(モデルの汚染)」**です。

そこで Plant Doctor は、「植物の『いつも』をその場で覚え、異常のときは覚えないエッジ AI」を実現するために、過去データを保持しない **オンデバイス逐次学習(Online Sequential Learning)** と、学習の前に置いた **7つの関門** を組み合わせています。本章ではその数理とファームウェア実装(`PlantAi.c` ・ `AiAnomaly.c`)を詳しく解説します。

4.1 データ非保持型逐次学習(ODL)とメモリ消費量 $O(1)$

バッチ学習と、Solist-AI™ が行うオンデバイス逐次学習(ODL)の違いを整理します。



比較項目	バッチ学習	Solist-AI™ オンデバイス逐次学習(ODL)
過去データの保存	必須(大容量ストレージが必要)	**不要(1件ごとに重みへ反映して即座に破棄)**
学習する場所	クラウドサーバー・PC	**鉢の横にある ML63Q2557 内蔵アクセラレータ(`AI_PERI`)**
必要な RAM 容量	数百 MB ～ 数十 GB	**数 KB(16 KB RAM のマイコンで常時稼働)**
鉢・環境への適応	固定(出荷時のまま)	**その株・鉢・土・照明の「いつも」を現場で学習**

1秒に1回サンプリングすると、1日で 86,400 件のデータが生まれます。ODL では新しいサンプルが1件届くたびに、その統計情報をモデルの内部行列へ織り込んでサンプル自体は捨てるため、**何日間連続で動かしても学習データのためにRAM が増えることはありません。**

4.2 逐次最小二乗法(RLS)と忘却係数 $\lambda = 1.0$

過去データを保存せずに学習を積み重ねられる理由は、出力層の重み更新に **逐次最小二乗法(Recursive Least Squares: RLS)** と **シャーマン・モリソンの公式(Sherman-Morrison Formula)** を使っているためです。

1. 保持する状態変数と更新式

隠れ層（64ノード）の出力ベクトルを $\mathbf{h}_t \in \mathbb{R}^{64}$、目標となる入力ベクトルを $\mathbf{x}_t \in \mathbb{R}^{8}$ としたとき、モデルが保持するのは **重み行列** $\mathbf{W}_t \in \mathbb{R}^{64\times 8}$ と、過去の特徴量の共分散情報を要約した **相関逆行列** $\mathbf{P}_t \in \mathbb{R}^{64\times 64}$ の2つだけです。新しいサンプルが届くと、次の3ステップで行列を更新します。

1. **ゲインベクトル $\mathbf{k}_t$ の算出:**

$$\mathbf{k}_t = \frac{\mathbf{P}_{t-1}\mathbf{h}_t}{\lambda + \mathbf{h}_t^T \mathbf{P}_{t-1}\mathbf{h}_t}$$

   分母がスカラーになるため、計算量の大きい逆行列演算（$O(N^3)$）を行わずに $O(N^2)$ で更新できます。

2. **重み行列 $\mathbf{W}_t$ の更新:**

$$\mathbf{W}_t = \mathbf{W}_{t-1} + \mathbf{k}_t(\mathbf{x}_t - \hat{\mathbf{x}}_t)^T$$

3. **相関逆行列 $\mathbf{P}_t$ の更新:**

$$\mathbf{P}_t = \frac{1}{\lambda}\left(\mathbf{P}_{t-1} - \mathbf{k}_t\mathbf{h}_t^T\mathbf{P}_{t-1}\right)$$

2. 忘却係数を $\lambda = 1.0$ に固定する理由

式中の λ（$0 < \lambda \leq 1$）は忘却係数です。$\lambda < 1.0$ にすると古い記憶を早く忘れるようになりますが、植物のように変化が穏やかな対象で $\lambda < 1.0$ のまま何日も学習を続けると、$\mathbf{P}_t$ が $1/\lambda$ 倍され続けて数値が発散する「共分散ワインドアップ（Covariance Wind-up）」が起きます。

そのため `PlantAi_Init` では、忘却係数を **`1.0`**（**`bfloat16`** で **`0x3F80`**）に設定し、長期間の連続運転でも数値が発散しないようにしています。

```
/* ai/PlantAi.c: Solist-AI 初期化パラメータ */
s_solistAiParams.inputSize = SOLIST_AI_FEATURE_DIM;  /* 8 */
s_solistAiParams.hiddenSize = 64U;                   /* 64 */
s_solistAiParams.outputSize = SOLIST_AI_FEATURE_DIM; /* 8 */
s_solistAiParams.forgettingFactor = 0x3F80;          /* 1.0 (bfloat16) */
s_solistAiParams.activationFunction = ODL_ACTV_SIGMOID;
s_solistAiParams.lossFunction = ODL_LOSS_MSE;
```

4.3 学習の進行段階と判定開始までの流れ

電源を入れてからの累積学習回数 `s_solistAiTrainCount` に応じて、システムは3つの段階（`s_solistAiPhase`）を進みます。同時に、第3章で解説した変化速度の履歴待機（15分）と誤差基準の学習（600件）が進みます。

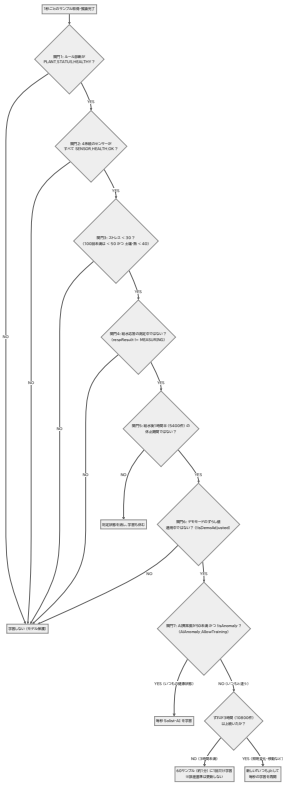



```
/* ai/PlantAi.c: 累積学習回数に応じたフェーズ更新 */
++s_solistAiTrainCount;
if (s_solistAiTrainCount < 100U) {
    s_solistAiPhase = 0U; /* PROFILING */
} else if (s_solistAiTrainCount < 500U) {
    s_solistAiPhase = 1U; /* STABILIZING */
} else {
    s_solistAiPhase = 2U; /* MONITORING */
}
```

1. **起動〜15分:** 1分ごとの履歴がまだ短いため、変化速度の特徴量（x_6, x_7）は中間値 `128` に固定して Solist-AI を学習させます。最初の100回（Phase 0）は立ち上げ時のわずかな偏りで学習が止まらないよう、土壌・熱の部分スコアが40未満なら総合ストレス50未満まで学習を許可します。
2. **15分〜約25分:** 変化速度の履歴が15分分たまり（`isRateReady == true`）、かつ学習回数が100回以上（Phase 1〜2）になると、`AiAnomaly_Update` が健康なときの再構成誤差の平均とばらつきを集め始めます。
3. **起動後約25分以降:** 健康なサンプルが **600件（約10分）** たまると基準が確立し（`AiAnomaly_IsReady == true`）、異常度 0〜100 の算出と `WARNING` 判定が始まります。

4.4 健康なモデルを汚さない「7つの関門」

オンデバイス学習で最も重要なのが、「いつ学び、いつ学ばないか」の制御です。`PlantAi.c` と `AiAnomaly.c` では、学習（`OSUAD_StartTrain`）の前に**7つの関門**を置き、健康なモデルが異常データや過渡状態によって汚されるのを防いでいます。



関門1〜4：ルール診断・センサー健全性・低ストレス・給水応答測定外

まずルール上の安全条件（`isRuleSafe`）として、①診断が `PLANT_STATUS_HEALTHY` であること、②土壌・葉温・温湿度・照度の4系統すべてが `SENSOR_HEALTH_OK` であること、③ストレススコアが `30` 未満（学習100回未満の立ち上げ時のみ、土壌・熱の部分スコアが `40` 未満なら `50` 未満）であること、④給水後5分間の効果測定中（`WATERING_RESPONSE_MEASURING`）ではないことを確認します。

関門5：給水後1時間半（5400サンプル）の休止

水やりが行われると、土壌水分は数分間で急激に上がります。ところが、Solist-AI に入力している土壌水分の変化速度（x_7）は**直近60分間の履歴**から計算されるため、土が湿り終わったあとも60分間は「土壌水分が猛烈な速さで増えている」という値が特徴量に残り続けます。想定値の計算で確かめたところ、給水直後に AI の判定や学習を動かすと、給水が成功（OK）したあとも約1時間にわたって再構成誤差が跳ね上がり、`WARNING` が出続けてしまうことが分かりました。

そこで、自動・手動を問わずポンプが動いたとき（`PlantAi_NotifyWatering`）には、**`PLANT_DOCTOR_AI_POST_WATERING_HOLD_SAMPLES = 5400`（1秒周期で5400サンプル＝1時間半）**の休止カウンタをセットします。

```
/* ai/PlantAi.c: 給水通知時に1時間半（5400サンプル）の休止を設定 */
void PlantAi_NotifyWatering(const PUMP_WATERING_EVENT *event, uint32_t currentTick) {
    (void)currentTick;
    WateringResponse_NotifyWatering(&s_wateringRespState, event, s_aiTick);
    /* 給水後は約1時間半、AIの判定と学習を休む（給水で直前の「いつもと違う」も解除する） */
    s_aiPostWateringHoldSamples = PLANT_DOCTOR_AI_POST_WATERING_HOLD_SAMPLES;
    AiAnomaly_ClearJudgement(&s_aiAnomaly);
}
```

この1時間半の間は、Solist-AI の学習を行わないだけでなく、`AiAnomaly_ClearJudgement` によって異常度（`score`）と `WARNING` 判定（`isAnomaly`）を `0` / `false` に保ちます。一方で、給水前までに学んだ**健康時の誤差の平均（`meanQ16`）**

とばらつき（`deviationQ16`）は消さずに残してあるため、1時間半が過ぎて変化速度の履歴が落ち着けば、すぐに元の基準で監視を再開できます（もちろん休止中も、給水失敗などのルール診断は休まず動いています）。

関門6：デモモードのオフセット補正中は学ばない

展示や動作確認で使うデモモード（DEMO2・DEMO3・DEMO5）では、スイッチやコンソール操作で土壌水分や葉温に擬似的なオフセット（ずらし値）を加えます。このオフセットはルール診断とストレス算出のコピー（`diagFeatures`・`diagHealth`）にだけ適用され、DEMO3 では実際に乾燥ストレスと判定されてポンプが動きます。しかし、オフセットが1つでも入っている間（`isDemoAdjusted == true`）は Solist-AI の学習を止めるため、デモ操作で AI のモデルや誤差基準が汚れることはありません。

関門7：異常時の学習間引き（1分に1回）と、3時間後の学び直し

関門1〜6をすべて通過して `isRuleSafe == true` になったとしても、ルールしきい値に達する前の「いつもと違う」変化が始まっているかもしれません。そこで最後に `AiAnomaly_AllowTraining`（`ai/AiAnomaly.c`）を通します。

```
/* ai/AiAnomaly.c: いつもと違う間は学習を絞り、3時間続いたら学び直す */
bool AiAnomaly_AllowTraining(AI_ANOMALY_STATE *state, const AI_ANOMALY_CONFIG *config, bool isRuleSafe) {
    if ((state == NULL) || (config == NULL) || !isRuleSafe) {
        return false;
    }
    if (!state->isAnomaly && (state->score < config->onScore)) {
        state->deviatingSamples = 0U;
        return true;
    }
    if (state->deviatingSamples < UINT16_MAX) {
        ++state->deviatingSamples;
    }
    if (state->deviatingSamples >= config->relearnAfterSamples) {
        return true;
    }
    return (config->relearnEverySamples > 0U) &&
           ((state->deviatingSamples % config->relearnEverySamples) == 0U);
}
```

- **平常時（`!isAnomaly` かつ `score < 50`）**: 毎秒（1サンプルごと）Solist-AI を学習させ、誤差の基準も更新します。
- **いつもと違う間（`score >= 50` または `isAnomaly == true`）**: 進行中の乾燥や異常を「いつも」として覚えてしまわないよう、誤差の基準更新を止め、Solist-AI の学習も `relearnEverySamples = 60`（**60サンプル＝約1分に1回**）だけに絞ります。
- **3時間（10800サンプル）続いたときの学び直し**: もしルール上は健康でストレスも30未満のまま、「いつもと違う」状態が `relearnAfterSamples = 10800`（**約3時間**）続いた場合は、植物の異常ではなく「照明の点灯・消灯」や「鉢の置き場所の変更」によって環境が切り替わったと判断し、新しい「いつも」に馴染ませるために毎秒の学習を再開します。
- **なぜ30分ではなく3時間なのか**: 第2章で述べたとおり、ストレスの重みを `35 / 35 / 15 / 15` にしたことで、土壌水分が 52% から 30% へ乾いていく途中でもストレススコアは 30 未満に保たれます。想定値の計算で検証したところ、学び直しまでの時間が30分（1800サンプル）だと、乾いている最中に30分が経過して「速い乾き」を新しい「いつも」として覚え直してしまい、`WARNING` が途中で消えてしまうことが分かりました。そこで学び直しまでの時間を **3時間（10800サンプル）** に延ばし、数時間かけて進む乾燥のあいだ `WARNING` を維持できるようにしています。

4.5 数値発散ウォッチドッグとタイムアウト時の安全停止

長期間の稼働中に強いノイズなどでアクセラレータ内部の演算が飽和したり、ハードウェアが応答しなくなったりした場合でも、システム全体の監視やポンプ制御を止めないための保護処理を入れています。

```
/* ai/PlantAi.c: 数値発散ウォッチドッグによるモデルと基準の初期化 */
if (fLoss >= 1.0f) {
    if (++s_solistAiDivergenceCount >= 10U) {
        OSUAD_Initialize(&s_solistAiParams, 1U);
        s_solistAiDivergenceCount = 0U;
        s_solistAiLatestLossPpm = 250U;
        /* モデルを作り直したら、学習の段階と異常度の基準も最初から作り直す */
        s_solistAiTrainCount = 0U;
        s_solistAiPhase = 0U;
        AiAnomaly_Reset(&s_aiAnomaly);
    }
} else {
    s_solistAiDivergenceCount = 0U;
}
```

1. **数値発散からの自己修復:** 学習可能な健康状態であるにもかかわらず、再構成誤差が最大値（`1.0f` = `10000 ppm`）に張り付いた状態が **10回連続** した場合は、`OSUAD_Initialize` で Solist-AI の重みを初期化します。このとき、累積学習回数 `s_solistAiTrainCount` と学習フェーズ `s_solistAiPhase` を `0` に戻し、`AiAnomaly_Reset` で異常度の基準も最初から作り直すことで、モデルと基準の食い違いを防ぎます。
2. **アクセラレータ応答タイムアウト時の安全停止:** `OSUAD_StartPredict` や `OSUAD_StartTrain` の完了待ちループ（上限 `50000` ループ、待機中もウォッチドッグをクリア）がタイムアウトした場合は、`s_solistAiInitialized = false` にして以降の AI 呼び出しを止めるとともに、`AiAnomaly_Reset(&s_aiAnomaly)` で異常度を `0` にリセットします。これにより、AI が止まったあともルール診断と自動給水はそのまま動き続けます。

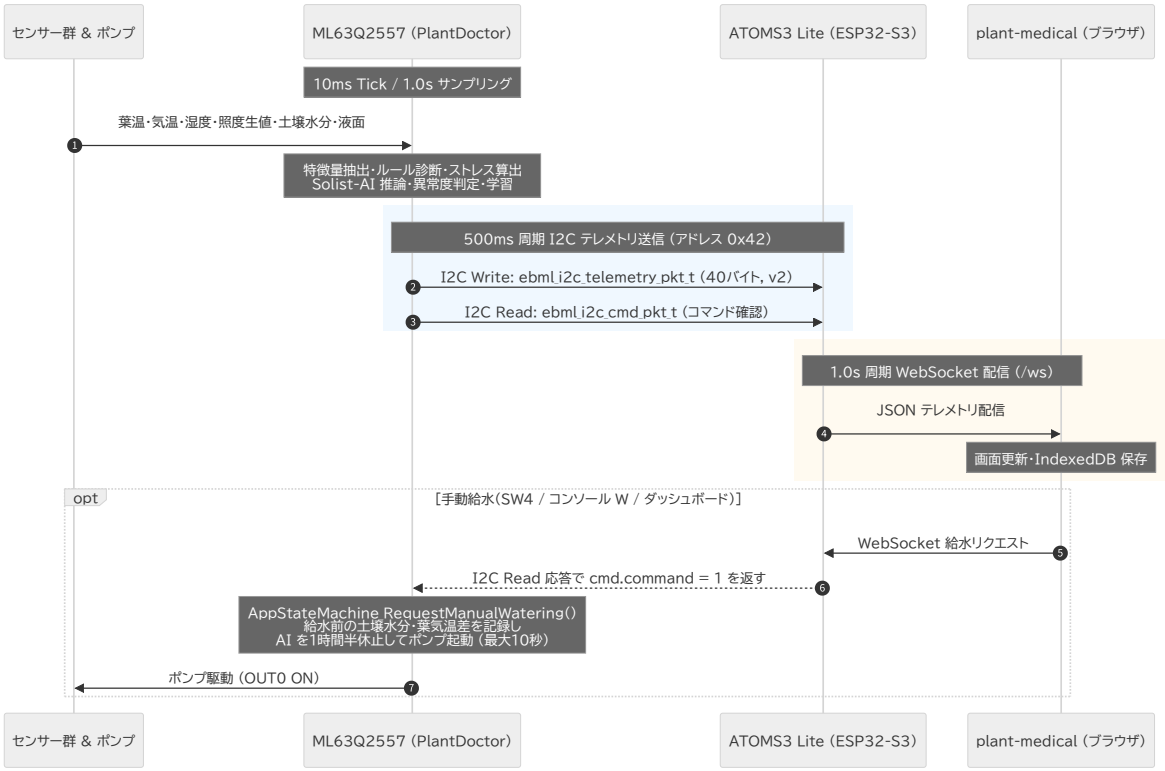
第5章 システム統合・通信テレメトリと生体コックピット

第1章から第4章までで、ROHM ML63Q2557 マイコンの内部で動くセンシング、ルール診断・ストレス算出、Solist-AI による異常度判定と7つの学習関門について解説しました。

Plant Doctor はマイコン基板1枚でも16×2 キャラクタ LCD とスイッチ(SW1～SW4)を使ってその場で状態確認と水やりができますが、さらに **ATOMS3 Lite(ESP32-S3)ゲートウェイ** と **Web ダッシュボード「plant-medical」**、および **UART コンソール** をつなぐことで、8次元特徴量や再構成誤差・異常度の推移をリアルタイムに観察したり、遠隔から手動給水を行ったりできます。本章では、これらを結ぶ通信プロトコルと UI の実装を詳しく解説します。

5.1 3層の連携と手動給水経路の統一

センサーの読み取りからブラウザの表示、そして手動給水の指示が戻るまでの流れは次のとおりです。



手動給水経路の統一(`AppStateMachine_RequestManualWatering`)

Plant Doctor には、人が手動でポンプを動かす方法が3つあります。

1. 基板上の **SW4** を押す
2. USB シリアル(UART コンソール)から `W` **コマンド** を送る
3. Web ダッシュボードの給水ボタンを押し、**ATOMS3 Lite 経由の I2C コマンド**(`cmd.command == 1U`) を受け取る

以前は、SW4 と自動給水ではポンプを動かす前に給水前の土壌水分と葉気温差を `PlantAi_NotifyWatering` へ記録していた一方、コンソールの `W` と ATOMS3 Lite からの給水指示は `PumpControl_RequestManualWatering` を直接呼んでいたため、給水前の値が記録されず、給水後の効果判定や AI の1時間半休止が働かない不整合がありました。

現在のファームウェアでは、コンソールと I2C テレメトリからの給水要求もすべて **`AppStateMachine_RequestManualWatering`**（**`app/AppStateMachine.c`**）に集約しています。

```
/* app/AppStateMachine.c: コンソール・ダッシュボードからの手動給水経路 */
uint8_t AppStateMachine_RequestManualWatering(void) {
    uint8_t res;

    if (s_state != APP_STATE_MONITOR) {
        return 3U;
    }
    res = PumpControl_RequestManualWatering();
    if (res == 0U) {
        AppStateMachine_NotifyWatering(false); /* 給水前の土壌水分・ΔTをAIへ通知し、1時間半休止を設定 */
        (void)PlantLog_RecordEvent(PLANT_LOG_EVENT_MANUAL_WATER_ON, 0U);
    }
    return res;
}
```

これにより、どの経路から給水しても、`MONITOR` 状態でのみ安全にポンプが動き、給水前の値にもとづく3分後・5分後の自己診断と、給水後1時間半の AI 休止が同じように働きます。

5.2 I2C バイナリテレメトリ（v2、40バイト）の仕様

ML63Q2557（`sensors/EbmlI2cTelemetry.c`）は 500 ms（50 ticks）ごとに、I2C スレーブとして動作する ATOMS3 Lite（7ビットアドレス `0x42`）へ 40 バイト固定長のバイナリパケット `ebml_i2c_telemetry_pkt_t` を送信します。ATOMS3 Lite がつながっていない場合は NACK で即座に復帰し、メインループを止めません。

```c
/* inc/PlantDoctorI2cProto.h: I2C テレメトリパケット定義 (v2, 40バイト) */
#define EBML_I2C_SLAVE_ADDR        0x42
#define EBML_I2C_PKT_MAGIC         0x5044   /* 'P', 'D' */
#define EBML_I2C_PKT_VERSION       2

#pragma pack(push, 1)
typedef struct {
    uint16_t magic;                        /* 0x5044 */
    uint8_t  version;                      /* 2 */
    uint8_t  length;                       /* 40 バイト */
    uint32_t sampleSequence;               /* サンプル番号 */
    uint16_t soilMoistureRaw;              /* 土壌水分生値 (0〜4095) */
    int16_t  leafTemperatureCentiC;        /* 葉面温度 [1/100 ℃] */
    int16_t  airTemperatureCentiC;         /* 気温 [1/100 ℃] */
    uint16_t relativeHumidityCentiPercent; /* 相対湿度 [1/100 %] */
    uint16_t illuminanceRaw;               /* 照度センサー生値 (※ルクスではなく生値。換算は ×0.0576 lx) */
    uint8_t  tankLiquidDetected;           /* タンク残水 (1:あり, 0:空) */
    uint8_t  pumpOn;                       /* ポンプ状態 (1:ON, 0:OFF) */
    uint8_t  stressScore;                  /* ストレススコア (0〜100) */
    uint8_t  statusCode;                   /* 診断ステータス (PLANT_STATUS) */
    uint8_t  soilTrendCode;                /* 土壌傾向 (SOIL_TREND) */
    uint16_t aiTrainCount;                 /* Solist-AI 累積学習回数 */
    uint16_t aiLossPpm;                    /* 最新再構成誤差 [ppm] (0〜10000) */
    uint8_t  aiPhase;                      /* 学習フェーズ (0:Profiling, 1:Stabilizing, 2:Monitoring) */
    uint8_t  aiAnomalyScore;               /* 異常度 (0〜100、ルール異常時は 85) */
    int16_t  leafTempRatePerHour;          /* 葉温変化速度 [1/100 ℃/h] */
    int16_t  soilMoistureRatePerHour;      /* 土壌水分変化速度 [%/h] */
    uint16_t checksum;                     /* 先頭38バイトの16-bit加算チェックサム */
} ebml_i2c_telemetry_pkt_t;
#pragma pack(pop)
```

1. 照度フィールド `illuminanceRaw` はセンサーの生値

第1章でも触れたとおり、パケット内の `illuminanceRaw` にはルクス（lx）換算後の値ではなく、**照度センサー（SEN0228 / VEML7700）の 16 ビット生値**が入っています。ファームウェア内の換算式では **生値 1 が約 0.0576 lx**（`illuminanceCentiLux = (raw * 576 + 50) / 100`）であり、実機ログの照明点灯中の値（約 `1350`）は約 78 lx にあたります。Web ダッシュボードやログの JSON では、この生値がそのまま表示・記録されます。

2. ルール診断が異常のときは異常度 `85` を送る理由

パケットの `aiAnomalyScore` には、通常は `AiAnomaly.c` が算出した Solist-AI の異常度（0 ～ 100）が入ります。しかし、Web ダッシュボード（`SolistAiStatusCard.tsx`）は `aiAnomalyScore` が 30 未満のときに「健全状態を維持」と表示する作りになっています。

一方、Solist-AI の異常度は、起動後の準備中や給水後1時間半の休止中、あるいは誤差が平常平均以下のときには `0` になります。そのため、もし `PlantAi_GetAiAnomalyScore()` の値をそのまま送ってしまうと、たとえば起動直後に土が乾いていてルール診断が `DRY_STRESS`（乾燥ストレス）や `SENSOR_ERROR`（センサー異常）を出しているのに、ダッシュボードの AI カードには「異常度 0・健全状態を維持」と表示されてしまいます。

これを防ぐため、`EbmlI2cTelemetry.c` では**診断ステータスが `HEALTHY`・`WARNING`・`LEARNING` 以外の異常状態にある間は、従来どおり異常度として `85` を上書き送信**します。

```
/* sensors/EbmlI2cTelemetry.c: ダッシュボード向けの異常度送信 */
/* ダッシュボードは30未満を「健全」と表示する。ルールが異常と判定している間は、従来どおり85を送る */
pkt.aiAnomalyScore = PlantAi_GetAiAnomalyScore();
if ((pkt.statusCode != (uint8_t)PLANT_STATUS_HEALTHY) && (pkt.statusCode != (uint8_t)PLANT_STATUS_WARNING)
&&
    (pkt.statusCode != (uint8_t)PLANT_STATUS_LEARNING)) {
    pkt.aiAnomalyScore = 85U;
}
```

これにより、ルールで明らかな異常(乾燥・熱・給水失敗・センサー異常など)が出ているときにはダッシュボードも確実に異常表示になり、ルール上は健康(`HEALTHY`)または AI 早期警告(`WARNING`)のときには Solist-AI が算出した 0〜100 の異常度がそのまま表示されます。

5.3 ローカル 16×2 LCD 表示・スイッチ操作・デモモード

DT-EBML63Q2557 ボード上の 16文字×2行 LCD(`ui/LcdUi.c`)と4つのスイッチ(SW1〜SW4)だけでも、すべての状態確認と手動操作が行えます。すべての表示文字列は 16 文字以内に収まるよう設計されています。

1. 3つの表示ページと5秒自動巡回

通常監視中(`MONITOR`)は5秒ごとにページ 0 → 1 → 2 が自動で切り替わり、表示中のページも1秒ごとに最新値で書き換えられます。SW1・SW2・SW3 を短押しすると、見たいページへ即座に切り替えられます。

ページ	スイッチ	1行目の表示例	2行目の表示例	内容
ページ 0	SW1	`STRESS: 18/100 *`	`HEALTHY` / `WARNING`	ストレスバロメータ(0〜100、デモ中は末尾に `*`)と診断ステータス(異常センサーがある場合は `SOIL SENS ERR` 等)
ページ 1	SW2	`LEAF-AIR:+0.5C`	`SOIL:STABLE`	葉気温差 ΔT($0.1^\circ\mathrm{C}$ 単位)と土壌傾向(`STABLE` / `DRY` / `WET` / `DEGRADE`)
ページ 2	SW3	`LUX:77.76`	`TANK:WET`	ルクス換算照度と給水タンク液面(`WET` / `EMPTY`)。デモモード中はデモ番号を表示

2. SW4 による手動給水と LCD メッセージ

- **給水停止中に SW4 短押し:** タンクに水がありクールダウン中でなければポンプを起動し、LCD 2行目に **`WATERING`** と表示します(ポンプは最大 **10.0秒＝約20 mL** で自動停止し、停止後 **3.0秒** は再起動しません。以前の `WATERING 2.0s` という誤表示は `WATERING` に修正しました)。
- **ポンプ動作中に SW4 短押し:** その場でポンプを緊急停止し、`PUMP STOPPED` と表示します。
- **インターロック作動時:** タンクが空なら `TANK EMPTY!` 、停止後3秒以内なら `PUMP COOLDOWN` を 1.5 秒間表示して操作をブロックします。

3. デモモード(SW1 3秒長押し / コンソール `M=`)

展示や発表で短時間に各状態の動きを見せるため、**SW1 を3秒以上長押し**(または UART コンソールで `M=1` 〜 `M=5` を入力)するとデモモードに入ります。デモモード中に SW2 または SW3 を押すと、DEMO1(通常)→ DEMO2(葉温 $+2.00^\circ\mathrm{C}$・照度2倍)→ DEMO3(土壌水分 $-400‰$)→ DEMO4(タンク空)→ DEMO5(土壌センサー異常)を順に切り替えられます。

```
/* app/AppStateMachine.c: デモのずらし値を PlantAi の診断・ストレス入力にも渡す */
DemoMode_GetInput(&s_demoState, &demoIn);
PlantAi_SetDemoAdjustment(demoIn.leafTemperatureOffsetCentiC,
                          demoIn.soilMoistureOffsetPermille,
                          demoIn.forceSoilSensorError);
```

デモモードのオフセットは自律給水ポリシー（`WateringPolicy`）だけでなく、`PlantAi` のルール診断とストレス算出にも渡されます。そのため、土が実際に湿っている状態でも **DEMO3 に切り替えれば診断が `DRY_STRESS` になり、実際にポンプが動いて自動給水の動作を実演できます。**一方で第4章で説明したとおり、デモのオフセットは Solist-AI の入力・学習には入らないため、デモを行ったあと通常モード（`M=0` または SW1 長押し）に戻せば、学習済みのモデルを汚さずにそのまま監視を続けられます。

5.4 UART コンソール（115200 bps）

PC と USB シリアルで接続すると、115200 bps のテキストコマンド（`console/Console.c`）で状態の問い合わせや操作が行えます。

コマンド	動作
`P`	保存されている FRAM ログの件数やヘッダー情報を表示
`D`	保存されているログを CSV 形式でダンプ出力（末尾の余分な空行なし）
`S`	現在の時刻、センサー値、ストレススコア、診断ステータスを1行で表示
`W`	手動給水を要求（`AppStateMachine_RequestManualWatering` 経由で給水前値を記録して起動）
`C=dry,wet`	土壌水分センサーの乾燥・湿潤校正値を設定（整数チェックおよび `wet >= dry + 200` を検証）
`T=epoch`	現在時刻（Unix 秒、または `YYYY-MM-DDTHH:MM:SS`、2099年まで対応）を設定
`M=0` ～ `5`	デモモードの解除（`0`）および DEMO1～DEMO5（`1` ～ `5`）への直接切り替え
`Q`	USB 接続時のダッシュボード連携用 JSON を1行出力（`status` 名は `PLANT_STATUS` の番号と一致）

5.5 ATOMS3 Lite と Web ダッシュボード「plant-medical」

ATOMS3 Lite（ESP32-S3）は I2C で受け取った 40 バイトのテレメトリをチェックサム検証のうえ JSON に変換し、Wi-Fi の WebSocket（`/ws`）で配信します。mDNS（`http://plant-doctor.local`）に対応しているため、同じネットワーク上の PC やスマートフォンから IP アドレスを調べずにアクセスできます。

ブラウザで動く **plant-medical** は、受け取った生体テレメトリをブラウザ内の **IndexedDB** に毎秒保存しながら、次の3つのビューで植物の状態を可視化します。

1. **コックピット画面（Cockpit）**: ストレスバロメータ（0～100）、現在の診断ステータス（`HEALTHY` / `WARNING` / `DRY_STRESS` など）、葉温・気温・ΔT・湿度・土壌水分・照度（生値）、および手動給水ボタンを表示します。
2. **Solist-AI 研究室（Solist-AI™ Research）**: 累積学習回数（`aiTrainCount`）、学習フェーズ（Profiling / Stabilizing / Monitoring）、再構成誤差（`aiLossPpm`）の時系列グラフ、異常度（`aiAnomalyScore`、0～100）、および正規化された8次元特徴量のレーダーチャートを表示します。
3. **履歴・エクスポート（History & Export）**: IndexedDB に蓄えた時系列データを振り返り、**Apache Parquet** または **CSV** 形式でダウンロードして手元の環境で詳しく分析できます。